



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES
MULTIPLATAFORMA

Departamento de Informática

PROYECTO



Eventvs Mérida

Manual Técnico

Autor/es: David Muñoz Collado, Adrián Pérez Morales, Eva Retamar Muñoz

Curso Académico: 2025/2026

Tutor Proyecto : María Francisca Roncero Holgado

Índice

Índice	1
1. Introducción	2
Descripción del proyecto	2
2. Objetivos	2
3. Tecnologías involucradas	3
3.1. Frontend móvil	3
3.2. Backend	5
3.3. Base de datos	6
3.4. Almacenamiento de imágenes	7
3.5. Panel web de administración	7
3.6. Landing page	8
3.7. Web de restablecimiento de contraseña	9
3.8. Despliegue	10
3.9. Relación con decisiones de arquitectura	11
4. Proceso de desarrollo	12
4.1 Análisis	12
4.2. Desarrollo (diagrama de secuencias,...)	21
4.3 Desarrollo	22
4.4 Prototipado en Figma	25
4.5 Arquitectura del repositorio	26
4.6 Desarrollo Iterativo	26
4.7 Pruebas realizadas	27
5. Bibliografía	32

1. Introducción

Descripción del proyecto

Eventvs Mérida es una aplicación móvil diseñada para centralizar la información sobre los eventos culturales y de ocio que se celebran en la ciudad de Mérida. Actualmente, gran parte de esta información se encuentra dispersa en diferentes páginas web, redes sociales o medios locales, lo que dificulta que ciudadanos y turistas puedan conocer de forma sencilla todas las actividades disponibles.

La plataforma está dirigida tanto a ciudadanos de Mérida como a turistas que desean conocer la oferta cultural de la ciudad. A través de una interfaz sencilla e intuitiva, los usuarios pueden explorar los eventos disponibles, visualizar su información y conocer su ubicación.

2. Objetivos

Objetivos funcionales

- Centralizar la información de eventos culturales de Mérida en una única aplicación.
- Permitir a los usuarios consultar un listado actualizado de eventos.
- Mostrar la información detallada de cada evento.
- Visualizar la ubicación de los eventos mediante un mapa.
- Permitir el registro y autenticación de usuarios en la aplicación.

Objetivos técnicos

- Desarrollar una aplicación móvil utilizando tecnologías actuales de desarrollo.
- Implementar un sistema de gestión de usuarios mediante autenticación.
- Integrar una base de datos en la nube para almacenar la información de eventos.
- Aplicar buenas prácticas de programación y diseño de interfaces.

3. Tecnologías involucradas

3.1. Frontend móvil

La aplicación móvil se ha desarrollado con Flutter y Dart.

Flutter nos ha permitido crear una aplicación móvil moderna y con una interfaz visual adaptable. Además, al ser una tecnología multiplataforma, permite que el proyecto pueda ampliarse en un futuro a otros entornos, como iOS o web, aunque en esta versión el objetivo principal sea Android.

En la aplicación móvil se han utilizado diferentes paquetes para cubrir funcionalidades concretas:

Tecnología / paquete	Uso en el proyecto
Flutter	Framework principal utilizado para desarrollar la aplicación móvil
Dart	Lenguaje de programación utilizado en Flutter
flutter_localizations	Soporte de localización para adaptar componentes como fechas, calendarios y selectores al idioma de la aplicación
http	Comunicación con la API REST del backend mediante peticiones HTTP
http_parser	Definición del tipo de contenido en peticiones multipart, especialmente para subir imágenes
path	Gestión de rutas, nombres y extensiones de archivos seleccionados desde el dispositivo

Proyecto “Desarrollo de Aplicaciones Multiplataforma”

Título del Proyecto: Eventvs Mérida



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

go_router	Gestión de rutas y navegación entre pantallas
shared_preferences	Persistencia local de datos sencillos, como usuario, autologin o estado del tutorial
flutter_secure_storage	Almacenamiento seguro de información sensible, como tokens de autenticación
flutter_map	Visualización de mapas
latlong2	Gestión de coordenadas geográficas mediante latitud y longitud
table_calendar	Visualización de calendario de eventos
intl	Formateo de fechas, horas e internacionalización
image_picker	Selección de imágenes desde la galería del dispositivo
url_launcher	Apertura de enlaces externos desde la aplicación
share_plus	Compartición de eventos mediante aplicaciones instaladas en el dispositivo
flutter_linkify	Detección de enlaces dentro de textos y conversión en enlaces pulsables

tutorial_coach_mark	Tutorial guiado inicial paa explicar el funcionamiento de la app
cupertino_icons	Iconos de estilo iOS/Cupertino disponible para la interfaz

3.2. Backend

El backend se ha desarrollado con Spring Boot, utilizando Java como lenguaje principal.

Spring Boot se ha elegido porque permite crear una API REST robusta y organizada, separando el proyecto por capas: controladores, servicios, repositorios, entidades, DTOs y mappers. Esta estructura facilita el mantenimiento, la ampliación de funcionalidades y la localización de errores.

En el backend se han utilizado principalmente:

Tecnología	Uso en el proyecto
Java	Lenguaje principal del backend
Spring Boot	Framework principal del backend
Spring Web	Creación de controladores y endpoints REST
Spring Data JPA	Acceso a base de datos mediante repositorios
Spring Security	Autenticación, sesiones, roles y permisos

Maven	Gestión de dependencias y construcción del proyecto
Lombok	Reducción de código repetitivo en entidades y servicios
BCryptPasswordEncoder	Cifrado de contraseñas
springdoc-openapi	Documentación de endpoints
RestClient	Comunicación con servicios externos como Supabase Storage

3.3. Base de datos

La base de datos utilizada es PostgreSQL, alojada en Supabase.

Supabase ha permitido disponer de una base de datos en la nube sin necesidad de mantener un servidor propio. Además, PostgreSQL es una base de datos relacional robusta y adecuada para el modelo de datos del proyecto, donde existen relaciones entre usuarios, roles, eventos, categorías y eventos guardados.

Tecnología	Uso en el proyecto
PostgreSQL	Base de datos relacional
Supabase	Plataforma cloud donde se aloja la base de datos
JPA/Hibernate	Mapeo entre entidades Java y tablas de base de datos

La base de datos almacena la información principal del sistema:

- Usuarios
- Roles
- Eventos
- Categorías
- Eventos guardados por usuario
- Tokens de recuperación de contraseña
- Tokens relacionados con autenticación

3.4. Almacenamiento de imágenes

Para almacenar imágenes se ha utilizado Supabase Storage.

No se guardan las imágenes directamente en la base de datos, ya que esto haría que la base de datos creciera innecesariamente y complicaría la gestión de archivos. En su lugar, las imágenes se almacenan en buckets de Supabase Storage y en la base de datos se guarda la URL o el path correspondiente.

Bucket	Uso
imagenesEvento	Imágenes asociadas a eventos
imagenesPerfil	Fotos de perfil de usuarios

3.5. Panel web de administración

El panel web de administración se ha desarrollado con HTML, CSS, JavaScript, Bootstrap y la plantilla SB Admin.

Este panel permite a los administradores gestionar usuarios, eventos, organizadores, categorías y roles desde una interfaz web. También incluye un

dashboard con contadores y gráficos para visualizar información general del sistema.

Tecnología	Uso en el proyecto
HTML	Estructura de las páginas
CSS	Estilos personalizados
JavaScript	Lógica del panel y consumo de la API
Bootstrap	Diseño responsive y componentes visuales
SB Admin	Plantilla base del panel de administración
Simple Datatables	Tablas dinámicas
Font Awesome	Iconos

El panel consume la misma API REST que la aplicación móvil, por lo que comparte backend, base de datos y lógica de negocio.

3.6. Landing page

La landing page del proyecto se ha desarrollado con Angular 19.

Su objetivo es presentar el proyecto, mostrar información general, permitir la descarga de la APK y facilitar el acceso al repositorio de GitHub.

Tecnología	Uso en el proyecto
Angular 19	Desarrollo de la landing page
TypeScript	Lenguaje utilizado en Angular
SCSS	Estilos de la landing
i18n	Traducciones en varios idiomas
Vercel	Despliegue de la landing page

La landing page incluye diseño responsive, animaciones, selector de idioma y secciones informativas sobre la aplicación y el equipo.

3.7. Web de restablecimiento de contraseña

La web de restablecimiento de contraseña se ha desarrollado con HTML, CSS y JavaScript.

Esta página permite que un usuario introduzca una nueva contraseña mediante un token recibido por URL. El token se envía al backend, donde se valida si es correcto, si ha caducado o si ya ha sido utilizado.

Tecnología	Uso en el proyecto
HTML	Estructura de la página
CSS	Diseño visual

JavaScript	Validación y envío de datos al backend
Bootstrap	Componentes visuales
SweetAlert2	Mensajes de éxito y error
i18n	Traducciones
Vercel	Despliegue de la página

3.8. Despliegue

Para el despliegue del proyecto se han utilizado diferentes plataformas cloud según el tipo de componente.

Componente	Plataforma de despliegue
Backend/API REST	Render
Landing page	Vercel
Panel web de administración	Vercel
Web de restablecimiento de contraseña	Vercel

Base de datos	Supabase
Almacenamiento de imágenes	Supabase Storage

El backend se ha desplegado en Render mediante Docker, mientras que las webs se han desplegado en Vercel. Supabase se utiliza como servicio cloud para base de datos y almacenamiento.

Esta organización permite que la aplicación pueda probarse desde un dispositivo móvil sin necesidad de ejecutar manualmente todos los servicios en local.

3.9. Relación con decisiones de arquitectura

Las principales decisiones tecnológicas del proyecto se documentaron mediante ADR. Estas decisiones justifican la elección de las tecnologías principales:

ADR-001: Selección de Flutter para el desarrollo del frontend móvil.

ADR-002: Selección de Spring Boot para el backend.

ADR-003: Selección de Supabase para la base de datos y autenticación.

ADR-004: Selección de Render para el despliegue del backend.

Estas decisiones han servido para mantener una línea tecnológica coherente durante el desarrollo y justificar por qué se eligieron determinadas herramientas frente a otras alternativas.

Tabla EVEN01: Usuarios

Descripción	La entidad Usuario almacena la información de las personas registradas en la plataforma. Estos usuarios pueden consultar eventos, participar en ellos o, dependiendo de su rol, gestionarlos.				
Estructura					
NOMBRE	DESCRIPCIÓN	TIPO	LONG.	DEC.	RESTRICCIÓN
ID	Identificador único del usuario	Numérico		0	
NOMBRE	Nombre del usuario	Alfanumérico	50		
APELLIDOS	Apellidos del usuario	Alfanumérico	150		
FECHA_NACIMIENTO	Fecha de nacimiento del usuario	Fecha			
EMAIL	Correo electrónico del usuario, usado para acceso o contacto	Alfanumérico	255		Único
TELEFONO	Número de teléfono del usuario	Alfanumérico	9		Único
PASSWORD	Contraseña para autenticación en la app	Alfanumérico	8 - 128		
ID_ROL	Referencia al rol del usuario dentro del sistema	Numérico		0	
FOTO_PATH	Nombre con el que está almacenado la imagen de perfil del usuario	Alfanumérico	100		Nulable

Tabla EVEN02: Rol

<i>Descripción</i>	La entidad Rol define los diferentes tipos de usuarios dentro de la plataforma.				
<i>Estructura</i>					
<i>NOMBRE</i>	<i>DESCRIPCIÓN</i>	<i>TIPO</i>	<i>LONG.</i>	<i>DEC.</i>	<i>RESTRICCIÓN</i>
<i>ID</i>	Identificador único del rol	<i>N Numérico</i>		0	
<i>NOMBRE</i>	Nombre del rol (por ejemplo: administrador, organizador, usuario)	<i>A Alfanumérico</i>	150		Único

Tabla EVEN03: Categoría

Descripción	La entidad Categoría permite clasificar los eventos según su tipo o temática.				
Estructura					
NOMBRE	DESCRIPCIÓN	TIPO	LONG.	DEC.	RESTRICCIÓN
ID	Identificador único de la categoría	Numérico		0	
NOMBRE	Nombre de la categoría (por ejemplo: conciertos, teatro, exposiciones, festivales)	Alfanumérico	150		Único

Tabla EVEN04: Evento

Descripción	La entidad Evento almacena toda la información relacionada con los eventos culturales disponibles en la plataforma.				
Estructura					
NOMBRE	DESCRIPCIÓN	TIPO	LONG.	DEC.	RESTRI.
ID	Identificador único del evento	Numérico		0	
TITULO	Nombre del evento	Alfanumérico	200		
DESCRIPCION	Información detallada sobre el evento	Alfanumérico	5000		
FECHA_HORA	Fecha y hora en la que se celebra	Fecha			
LOCALIZACION	Lugar donde se realiza el evento	Alfanumérico	255		
FOTO	Imagen representativa del evento	Alfanumérico	300		
ID_USUARIO	Usuario que creó o gestiona el evento	Alfanumérico		0	
ID_CATEGORIA	Categoría a la que pertenece el evento	Numérico			
LATITUD	Latitud del evento	Decimal		6	Nulable
LONGITUD	Longitud del evento	Decimal		6	Nulable

Tabla EVEN05: Usuario_Evento

Descripción	La entidad Usuario_Evento es una tabla intermedia que representa la relación entre usuarios y eventos.				
Estructura					
NOMBRE	DESCRIPCIÓN	TIPO	LONG.	DEC.	RESTRICCIÓN
ID_USUARIO	Identificador del usuario	Numérico		0	
ID_EVENTO	Identificador del evento	Numérico		0	

Tabla EVEN06: refresh_tokens

Descripción	La entidad refresh_tokens permite refrescar la cookie o "JSESSIONID" de un usuario logueado para mantener su sesión iniciada y que pueda realizar acciones sobre los diferentes endpoints que tenga permiso.				
Estructura					
NOMBRE	DESCRIPCIÓN	TIPO	LONG.	DEC.	RESTRICCIÓN
ID	Identificador único del token generado	Numérico		0	
ID_USUARIO	Identificador del usuario	Numérico		0	
TOKEN	Token generado para validar el cambio de contraseña	Alfanumérico	255		Único
EXPIRACION	Datetime con la fecha de expiración del token (30 minutos después de la creación)	Fecha			
USADO	Valor booleano que controla si el token ya ha sido utilizado o no	Booleano			

Tabla EVEN07: password_reset_tokens

Descripción	La entidad password_reset_tokens permite generar tokens para poder restablecer la contraseña.				
Estructura					
NOMBRE	DESCRIPCIÓN	TIPO	LONG.	DEC.	RESTR.
ID	Identificador único del token generado	N Numérico		0	
EXPIRACION	Datetime con la fecha de expiración de la cookie (2 horas después de la creación)	F Fecha			
REVOCADO	Valor booleano que controla si la cookie ya ha finalizado su tiempo de vida	B Booleano			
TOKEN	Token generado para la cookie	A Alfanumérico	255		Único
ID_USUARIO	Identificador del usuario el cuál se restablece la cookie	N Numérico		0	

4.1.2 Diagramas de casos de uso

4.1.2.1 Diagrama de usuario no registrado

Este diagrama muestra de forma resumida las funcionalidades disponibles que puede realizar un usuario que no se ha registrado en la aplicación.

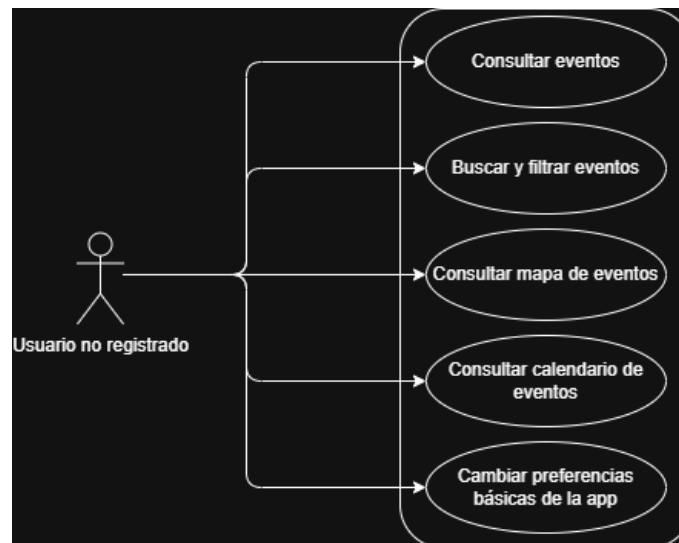


Figura 13: Diagrama de caso de uso de usuario no registrado

4.1.2.2 Diagrama de usuario registrado

Este diagrama muestra de forma resumida las funcionalidades que puede realizar un usuario registrado.

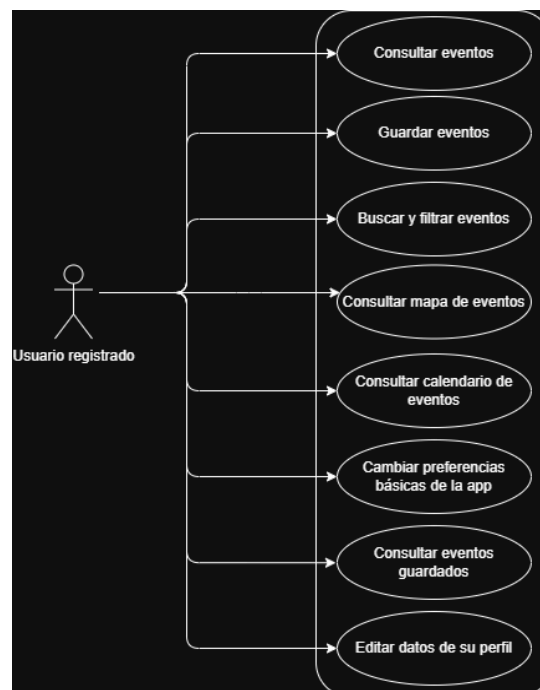


Figura 14: Diagrama de caso de uso de usuario registrado

4.1.2.3 Diagrama de organizador

Este diagrama muestra de forma resumida las funcionalidades del organizador.

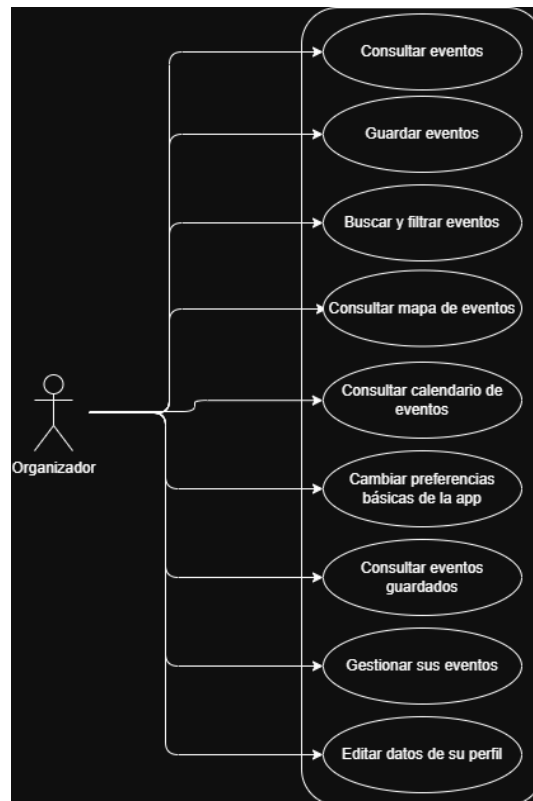


Figura 15: *Diagrama de caso de uso de organizador*

4.1.2.4 Diagrama de administrador

Este diagrama muestra de forma resumida las funcionalidades del administrador.

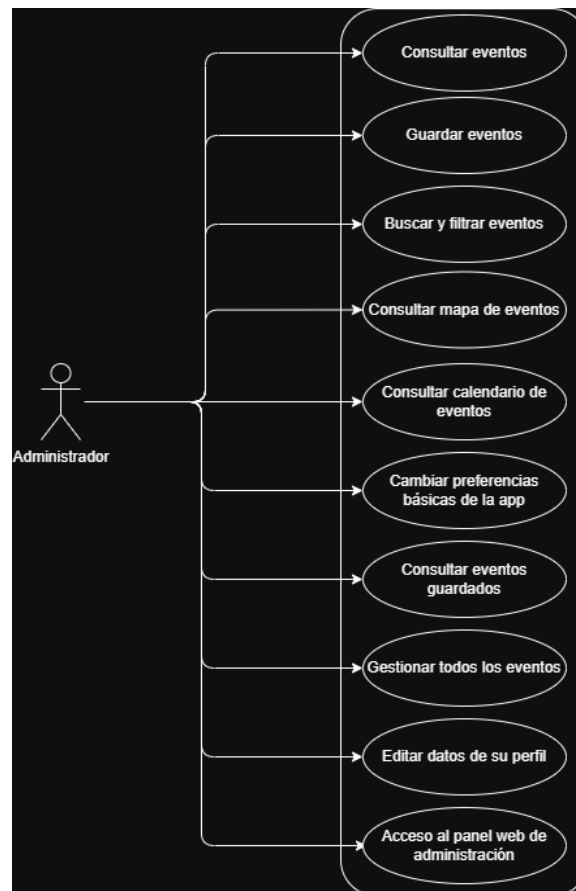


Figura 16: *Diagrama de caso de uso de administrador*

4.1.3 Diagrama de flujo

Este diagrama muestra el ciclo de vida que sigue la aplicación desde su inicio hasta el final.

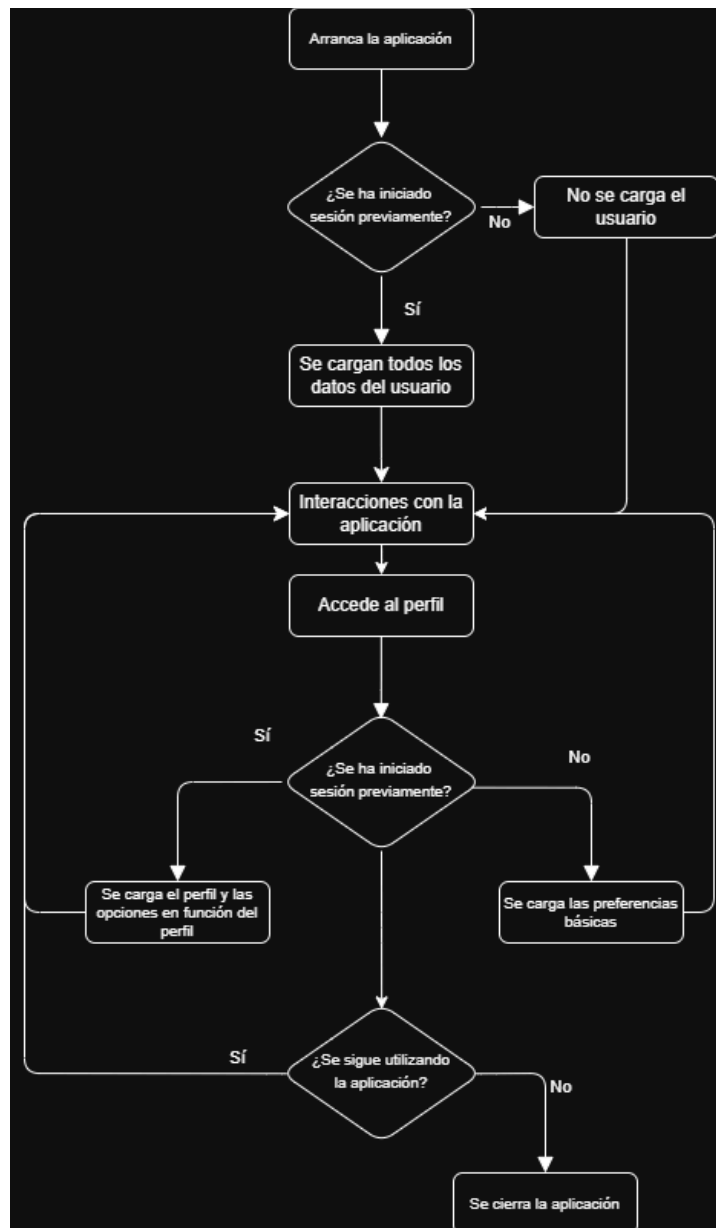


Figura 18: Diagrama de flujo

4.2. Desarrollo (diagrama de secuencias,...)

4.2.1 Diagrama de secuencias

Este diagrama muestra cómo funcionan todas las comunicaciones/acciones de la aplicación.

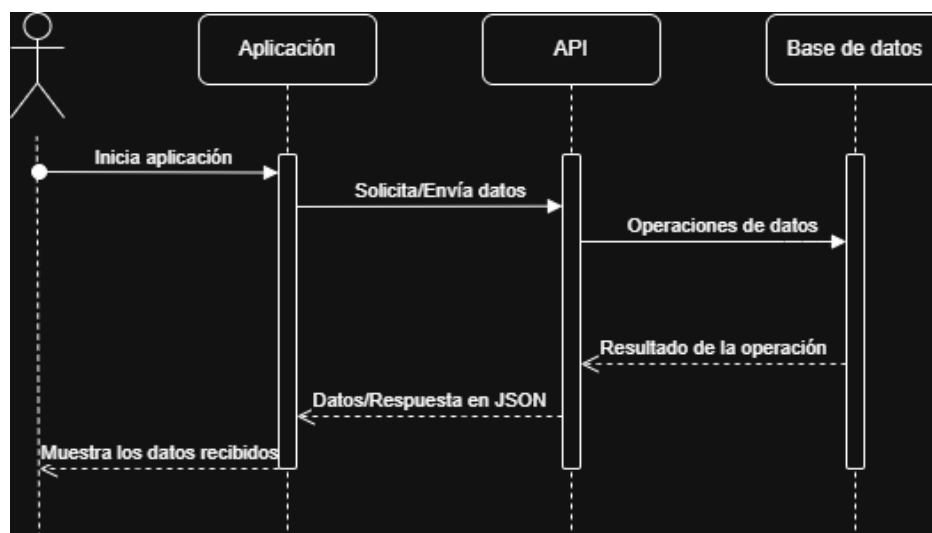


Figura 17: Diagrama de secuencias

4.3 Desarrollo

4.3.1 Planificación y Definición de Requisitos

La fase inicial consistió en identificar el problema a resolver: la fragmentación de la información cultural y de ocio de Mérida. A partir de este diagnóstico se definieron los requisitos principales del sistema:

- Consulta de eventos culturales y de ocio en Mérida.
- Filtrado por categoría, fecha y ubicación.
- Visualización de detalle de cada evento.
- Diseño accesible y atractivo para distintos perfiles de usuario (ciudadanos y turistas).
- Arquitectura cliente-servidor con API REST.

La definición del alcance se materializó en un prototipo elaborado en Figma, que sirvió de guía visual para basarnos a lo largo del desarrollo.

4.3.2 Diseño de la Arquitectura

La arquitectura del sistema de Eventvs Mérida se ha diseñado siguiendo una estructura en capas, con el objetivo de separar de forma clara la interfaz de usuario, la lógica de negocio y el acceso a los datos. Esta organización facilita el mantenimiento del proyecto, mejora su escalabilidad y permite que cada parte del sistema evolucione de forma independiente.

La solución está compuesta por una aplicación móvil desarrollada en Flutter, un backend implementado con Spring Boot y una base de datos gestionada mediante Supabase. Esta distribución permite construir un sistema modular en el que el frontend se encarga de la interacción con el usuario, el backend de la lógica de negocio y la exposición de servicios, y la capa de datos del almacenamiento y recuperación de la información.

4.3.2.1 Patrón de Arquitectura

El proyecto adopta una arquitectura en capas con un enfoque próximo a Clean Architecture, separando las responsabilidades principales del sistema en tres niveles: presentación, lógica de negocio y datos.

Esta elección se ha realizado porque permite:

- Mantener el código más organizado y modular.
- Separar la interfaz visual de la lógica interna de la aplicación.
- Facilitar el mantenimiento y la ampliación futura del sistema.
- Mejorar la reutilización de componentes.
- Simplificar las pruebas y la detección de errores.

En el caso de Eventvs Mérida, este enfoque resulta adecuado porque la aplicación no solo muestra información, sino que también necesita gestionar usuarios, consultar eventos, comunicarse con un backend y acceder a servicios externos. Por ello, una arquitectura en capas ofrece una solución más escalable que otros patrones más simples como MVC.

4.3.2.2 Diagrama de Arquitectura

La arquitectura general del sistema puede representarse de la siguiente forma:

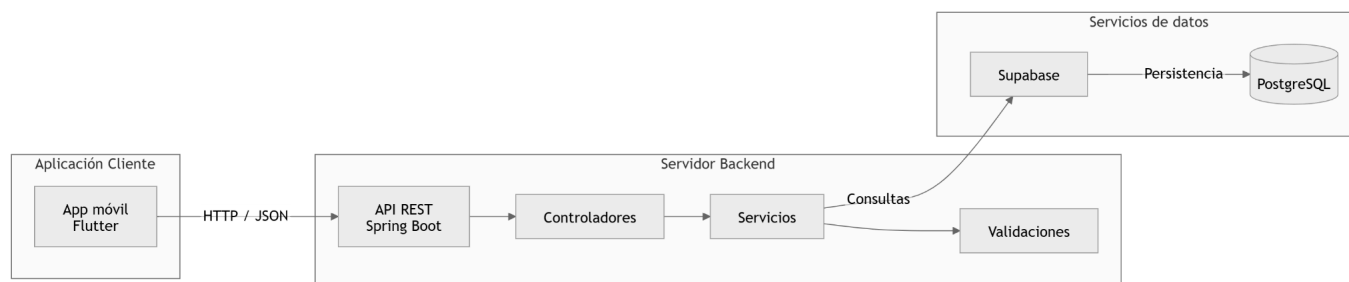


Figura 18: Diagrama de arquitectura

4.3.2.3 Componentes Principales

El sistema se organiza en tres capas principales: presentación, lógica de negocio y datos. Cada una de estas capas tiene responsabilidades específicas que permiten mantener una arquitectura modular y fácil de mantener.

Capa de presentación

La capa de presentación está formada por la aplicación móvil desarrollada en Flutter. Su función principal es mostrar la información al usuario y gestionar la interacción con la interfaz.

Entre los elementos principales de esta capa se encuentran:

- Pantallas de la aplicación (login, registro, listado de eventos, detalle de evento, mapa, etc.).
- Widgets reutilizables que componen la interfaz gráfica.
- Navegación entre pantallas.
- Formularios de entrada de datos.

Esta capa se encarga de recoger las acciones del usuario y enviar las peticiones correspondientes al backend a través de la API, pero no implementa la lógica principal del sistema.

Capa de lógica de negocio

La capa de lógica de negocio se encuentra implementada en el backend desarrollado con Spring Boot. Esta capa se encarga de procesar la información recibida desde la aplicación móvil, aplicar las reglas del sistema y coordinar el acceso a los datos.

Entre sus responsabilidades se encuentran:

- Gestión de usuarios y autenticación.
- Validación de datos recibidos desde la aplicación.
- Procesamiento de peticiones relacionadas con eventos.
- Implementación de reglas de negocio del sistema.
- Exposición de una API REST que permite la comunicación entre la aplicación móvil y el backend.

Esta capa actúa como intermediaria entre la interfaz de usuario y la base de datos.

Capa de datos

La capa de datos es la responsable del almacenamiento y recuperación de la información utilizada por el sistema. En este proyecto se utiliza Supabase, que proporciona una base de datos basada en PostgreSQL.

Los principales componentes de esta capa son:

- Base de datos PostgreSQL, donde se almacenan los eventos, usuarios y demás información de la aplicación.
- Repositorios de acceso a datos, utilizados por el backend para consultar o modificar la información.
- Servicios de base de datos proporcionados por Supabase, que facilitan la gestión de datos y autenticación.

Esta capa permite persistir la información del sistema y garantizar que los datos puedan ser consultados o actualizados de forma segura por el backend.

4.4 Prototipado en Figma

Antes de comenzar el desarrollo de la interfaz, elaboramos un prototipo completo en Figma. Este paso fue fundamental para:

- Acordar la identidad visual del producto (paleta de colores, tipografía, iconografía).
- Validar los flujos de navegación principales antes de escribir código.
- Servir como referencia de diseño durante la implementación en Flutter.
- Reducir el número de revisiones de UI durante el desarrollo.

El prototipo refleja las pantallas principales: listado de eventos, detalle de evento, filtros y pantalla de inicio.

4.5 Arquitectura del repositorio

El repositorio se encuentra alojado en GitHub. La estructura de directorios adoptada separa claramente las distintas partes del proyecto:

/app → Código de la app en Flutter y backend en Spring Boot.

/docs → Documentación del proyecto.

/scripts → Scripts en Python auxiliares y de automatización.

/web → Código fuente de las 3 webs del proyecto.

4.6 Desarrollo Iterativo

El desarrollo se organizó en iteraciones cortas, abordando funcionalidades de forma incremental. Las principales áreas de trabajo en cada iteración fueron:

4.6.1 Backend (Spring Boot / Java):

- Definición del modelo de datos de eventos, categorías y ubicaciones.
- Implementación de endpoints REST para consulta y filtrado de eventos.
- Configuración de la capa de persistencia.
- Pruebas de integración de la API.

4.6.2 Frontend móvil (Flutter / Dart):

- Configuración inicial del proyecto Flutter y estructura de carpetas.
- Implementación de las pantallas principales siguiendo el prototipo Figma.
- Integración con la API REST del backend mediante peticiones HTTP.
- Gestión de estado y navegación entre pantallas.
- Adaptación del diseño a distintos tamaños de pantalla.

4.6.3 Landing Page (Angular / SCSS / Typescript):

- Maquetación y diseño responsivo de la página de presentación.
- Implementación de secciones: descripción del producto, capturas, equipo y enlace de descarga.

4.6.4. Panel de administración (HTML / CSS / JS / Bootstrap)

- Maquetación y diseño de las diferentes páginas del panel.
- Conexión de cada acción de cada página con los endpoints de la API.

4.6.5 Web de recuperación de contraseña (HTML / CSS / JS / Bootstrap)

- Maquetación y diseño del formulario de recuperación de contraseña.
- Implementación de la lógica para el restablecimiento de la contraseña.
- Configuración del envío de correos electrónicos para poder acceder a la web con un token válido.

4.6.6 Scripts auxiliares (Python):

- Automatización de tareas como la subida de eventos de algunas fuentes.
- Procesamiento y preparación de datos para poblar la base de datos.

4.7 Pruebas realizadas

Las pruebas que hemos realizado han sido de carácter manual. adoptando un enfoque continuo e incremental: cada vez que se completaba el desarrollo de una funcionalidad, probamos todos los posibles casos de uso antes de integrar el código en la rama principal.

Esto nos permitió detectar errores cuando el código aún es reciente y es más fácil localizarlo, evita la acumulación de errores evitando que se mezclaran con nuevas funcionalidades manteniendo una coherencia y rigurosidad al código.

Momento de prueba	Al definir las entidades (Evento, Categoría, Ubicación, etc)
Tipo de prueba	Prueba de integración manual del backend.
Qué se verificó	Los endpoints de creación de registros devolvían respuesta 201 con los datos correctamente persistidos. Los endpoints de consulta devolvían los registros almacenados en formato JSON esperado. Los errores de validación (campos obligatorios vacíos, tipos incorrectos) devolvían códigos de error adecuados (400). Los scripts Python de carga de datos poblaban la base de datos sin errores.

Criterio de éxito	La API respondía correctamente a las operaciones CRUD básicas sobre las entidades del dominio.
--------------------------	--

Momento de prueba	Al implementar los endpoints de listado y detalle de eventos de forma paginada.
Tipo de prueba	Prueba funcional manual haciendo uso de Swagger.
Qué se verificó	GET /eventos devolvía el listado completo de eventos con la estructura JSON correcta. GET /eventos/{id} devolvía el detalle de un evento existente. GET /eventos/{id} con un id inexistente devolvía error 404. Los campos de respuesta (título, descripción, fecha, categoría, ubicación) se correspondían con los datos almacenados. La paginación del listado funcionaba correctamente al variar los parámetros page y size.
Criterio de éxito	Todos los endpoints respondían con los datos y códigos HTTP correctos en los escenarios positivos y negativos probados.

P-03. API REST — Filtrado de eventos

Momento de prueba	Al implementar los parámetros de filtrado por categoría, fecha y ubicación.
Tipo de prueba	Prueba funcional manual con herramienta de test de APIs.
Qué se verificó	El filtro por categoría devolvía únicamente eventos de la categoría indicada. El filtro por rango de fechas devolvía solo eventos dentro del intervalo especificado. La combinación de varios filtros simultáneos producía resultados coherentes. Una consulta con filtros que no coincidían con ningún evento devolvía una lista vacía (no un error).
Criterio de éxito	El sistema filtraba correctamente los eventos según los parámetros recibidos, incluyendo combinaciones de filtros.

P-04. Desarrollo de las pantallas/componentes de la app (Flutter)

Momento de prueba	A medida que se implementaban las pantallas: listado de eventos , detalle de eventos, mapa, calendario, perfil, etc.
Tipo de prueba	Prueba de interfaz manual en emulador Android y, cuando fue posible, en dispositivo físico.
Qué se verificó	La pantalla de inicio se cargaba correctamente y mostraba los elementos de navegación esperados. El listado de eventos se mostraba con los datos reales obtenidos de la API. Cada tarjeta de evento mostraba correctamente título, fecha, categoría e imagen. La pantalla de detalle se abría al pulsar sobre un evento y presentaba toda la información completa. La app mostraba un indicador de carga (loading) mientras esperaba la respuesta del servidor. Ante una respuesta de error del servidor, la app mostraba un mensaje de error en lugar de bloquearse.
Criterio de éxito	La navegación entre pantallas era fluida, los datos se cargaban desde la API y el diseño era consistente con el prototipo.

P-05. Integración app móvil — backend (consumo de la API)

Momento de prueba	Al conectar la aplicación Flutter con los endpoints reales del backend.
Tipo de prueba	Prueba de integración manual end-to-end.
Qué se verificó	La app obtenía y mostraba correctamente los datos devueltos por la API en el listado de eventos. Los filtros aplicados en la interfaz se traducían en peticiones correctas a la API y los resultados mostrados eran los esperados. La app mostraba un indicador de carga (loading) mientras esperaba la respuesta del servidor. Ante una respuesta de error del servidor, la app mostraba un mensaje de error en lugar de bloquearse. El comportamiento era correcto tanto con backend local (desarrollo) como con la URL de despliegue.
Criterio de éxito	El flujo completo usuario → app → API → base de datos → respuesta funcionaba de extremo a extremo sin errores visibles.

P-06. Funcionalidad de filtrado en la interfaz de usuario

Momento de prueba	Al implementar los controles de filtrado en la app Flutter (selección de categoría, fechas, etc.).
Tipo de prueba	Prueba funcional manual en emulador y dispositivo físico.
Qué se verificó	Los controles de filtrado se mostraban correctamente en la interfaz. Al seleccionar un filtro, la lista de eventos se actualizaba mostrando solo los resultados correspondientes. Al eliminar o resetear un filtro, la lista volvía a mostrar todos los eventos. La combinación de varios filtros producía los resultados correctos. La interacción con los controles era fluida y no generaba retardos perceptibles.
Criterio de éxito	El sistema de filtrado funcionaba de forma correcta y ofrecía una experiencia de usuario ágil y coherente.

P-07. Panel web de administración

Momento de prueba	Al desarrollar las pantallas del panel de administración.
Tipo de prueba	Prueba visual manual en navegadores y en dispositivos móviles.
Qué se verificó	Todas páginas del panel se renderizaban correctamente y se conectaban con los diversos endpoints de la API.. Los enlaces de navegación interna funcionaban correctamente. El despliegue automático en Vercel reflejaba los últimos cambios del repositorio. Las imágenes y recursos se cargaban sin errores.
Criterio de éxito	El panel era completamente funcional en las conexiones con la API y visualmente consistente en todos los navegadores.

P-08. Landing page

Momento de prueba	Al desarrollar las distintas secciones de la página web informativa.
Tipo de prueba	Prueba visual manual en navegadores y en dispositivos móviles.
Qué se verificó	Todas las secciones de la página se renderizaban correctamente. El diseño era responsivo y se adaptaba a pantallas móviles y de escritorio. Los enlaces de navegación interna funcionaban correctamente. El despliegue automático en Vercel reflejaba los últimos cambios del repositorio. Las imágenes y recursos estáticos se cargaban sin errores.
Criterio de éxito	La landing page era completamente funcional y visualmente consistente en todos los navegadores y dispositivos probados.

F-09. Web de recuperación de contraseñas

Momento de prueba	Al desarrollar el formulario de reseteo de la contraseña.
Tipo de prueba	Prueba visual manual en navegadores y en dispositivos móviles.
Qué se verificó	Todos los campos del formulario se renderizaban correctamente. El diseño era responsivo y se adaptaba a pantallas móviles y de escritorio. La lógica con el token a la hora de hacer el reseteo de contraseña es válida y funcional. El despliegue automático en Vercel reflejaba los últimos cambios del repositorio. Las imágenes y recursos estáticos se cargaban sin errores.
Criterio de éxito	La web de recuperación de contraseña era completamente funcional y visualmente consistente en todos los navegadores y dispositivos probados.

5. Bibliografía

Proyecto “Desarrollo de Aplicaciones Multiplataforma”

Título del Proyecto: Eventvs Mérida



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

- Documentación oficial de Angular: <https://angular.dev/overview>
- Documentación oficial de Bootstrap: <https://getbootstrap.com/>
- Documentación oficial de Flutter: <https://docs.flutter.dev/>
- Documentación oficial de Python: <https://docs.python.org/3/>
- Documentación oficial de Spring Boot: <https://docs.spring.io/spring-boot/index.html>
- Documentación oficial de Supabase: <https://supabase.com/docs>
- Documentación Tutorial Coach Mark: https://pub.dev/packages/tutorial_coach_mark
- Resolución de dudas respecto a JS: <https://stackoverflow.com/questions>